

DATA STRUCTURE

LAB PROGRAM

23. Implementation of Shortest Path Algorithms using Dijkstra's Algorithm

```
#include <stdio.h>
```

```
#define V 5
```

```
#define INF 9999
```

```
int minDistance(int dist[], int visited[])
```

```
{
```

```
    int min = INF, min_index;
```

```
    for (int v = 0; v < V; v++)
```

```
    {
```

```
        if (visited[v] == 0 && dist[v] < min)
```

```
        {
```

```
            min = dist[v];
```

```
            min_index = v;
```

```
        }
```

```
    }
```

```
    return min_index;
```

```
}
```

```
void dijkstra(int graph[V][V], int source)
```

```
{
```

```
    int dist[V];
```

```
    int visited[V];
```

```
    for (int i = 0; i < V; i++)
```

```
    {
```

```
        dist[i] = INF;
```

```
        visited[i] = 0;
```

```
    }
```

```
    dist[source] = 0;
```

```
    for (int count = 0; count < V - 1; count++)
```

```
    {
```

```
        int u = minDistance(dist, visited);
```

```
        visited[u] = 1;
```

```
        for (int v = 0; v < V; v++)
```

```
        {
```

```
            if (!visited[v] &&
```

```
graph[u][v] &&
dist[u] != INF &&
dist[u] + graph[u][v] < dist[v])
{
    dist[v] = dist[u] + graph[u][v];
}
}
}
```

```
printf("Vertex\tDistance from Source\n");
for (int i = 0; i < V; i++)
{
    printf("%d\t%d\n", i, dist[i]);
}
}
```

```
int main()
{
    int graph[V][V] = {
        {0, 10, 0, 30, 100},
        {10, 0, 50, 0, 0},
        {0, 50, 0, 20, 10},
        {30, 0, 20, 0, 60},
```

```
    {100, 0, 10, 60, 0}  
};  
  
int source = 0;  
  
printf("Shortest Paths using Dijkstra's Algorithm\n");  
dijkstra(graph, source);  
  
return 0;  
}
```

Output

```
Shortest Paths using Dijkstra's Algorithm  
Vertex Distance from Source  
0      0  
1      10  
2      50  
3      30  
4      60  
  
=== Code Execution Successful ===
```